30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 4

Kotlin Coroutines & Concurrency — Structured Concurrency, Cancellation & Thread Safety



Esmaeeil Moradi
@esmaeeil-moradi700

Why coroutines are Day 4

Every modern Android API is asynchronous — and in 2026 it speaks `suspend`. Interviewers rarely ask *"what is a coroutine?"* anymore; they ask what happens when it **fails, gets cancelled or races**:

- Room, DataStore, WorkManager (`CoroutineWorker`), Paging, Ktor and Retrofit expose `suspend` and `Flow` APIs
- **Structured concurrency** replaces manual threads and callbacks — no leaks by design
- **Main-safety** is the convention: any suspend function is safe to call from the main thread
- Swallowed cancellation, lost exceptions and data races are classic senior filters
- `kotlinx.coroutines` is multiplatform — the same code runs in KMP modules

Level	Questions
Junior	suspend vs blocking · Dispatchers & main-safety · launch vs async
Mid	Scope & context · Android scopes · Cancellation · Exceptions
Senior	coroutineScope vs supervisorScope · Bounded parallel work · Testing
Architect	Error strategy · Shared mutable state · Under the hood



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: a tree of coroutines

```
viewModelScope   SupervisorJob + Dispatchers.Main.immediate
 ├── launch { loadProfile() }
 │     └── coroutineScope
 │           ├── async { api.user() }
 │           └── async { api.feed() }
 └── launch { observeCart() }
```

- **Cancellation flows down:** cancel a scope and every coroutine under it is cancelled
- **Failure flows up:** a failing child cancels its parent and siblings — unless the parent is a supervisor
- **A parent waits:** a coroutine completes only when all its children have completed

THE ONE QUESTION BEHIND EVERY COROUTINE ANSWER

Who is this coroutine's parent? If you can't name the scope that will cancel it, it's a leak.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 4

Junior

suspend vs blocking · Dispatchers & main-safety · launch vs async



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Suspending vs blocking

	Suspending	Blocking
The thread while waiting	Released — runs other work	Stuck until the call returns
Examples	`delay()`, Room / Retrofit `suspend` calls, `withContext`	`Thread.sleep()`, `InputStream.read()`, `runBlocking`
On the main thread	UI stays responsive	Jank, then ANR (Day 2)
Cancellation	Reacts at suspension points	Ignores coroutine cancellation

A coroutine is a **suspendable computation**: at each suspension point it can pause, give its thread back, and later resume — possibly on a different thread.

INTERVIEW TRAP

"*`runBlocking` is fine in production code.*" — On Android it blocks the thread it runs on; on the main thread that's a frozen UI. It belongs in `main()` functions and a few bridging cases — not in app code, and not in tests (use `runTest`).



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What `suspend` really means

- `suspend` means *"this function may pause without blocking"*. It can only be called from a coroutine or another suspend function.
- It does **not** pick a thread. The body runs on the caller's dispatcher until something inside switches with `withContext`.

```
// Called from viewModelScope → runs on the main thread
suspend fun loadAvatar(url: String): Bitmap {
    val bytes = api.download(url)   // suspends: thread is free
    // CPU work: still on the caller's thread (main!)
    return BitmapFactory.decodeByteArray(bytes, 0, bytes.size)
}
```

INTERVIEW TRAP

"Marking a function `suspend` moves it off the main thread." — **No.** A suspend function that decodes, parses or calls blocking I/O still blocks whichever thread called it. Fix it inside the function: `withContext(Dispatchers.Default) { … }`.

INTERVIEW TRAP

"A coroutine is a lightweight thread." — A useful analogy, a wrong model. A coroutine is a task scheduled **on** threads; after a suspension it may resume on another thread of the same dispatcher, so `ThreadLocal` values don't follow it.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Dispatchers — which threads run your code

Dispatcher	Use for	Worth knowing
`Main`	UI, calling main-safe suspend functions	The Android main thread
`Main.immediate`	Same, without re-dispatching if already on main	Default of `viewModelScope` and `lifecycleScope`
`Default`	CPU work: sorting, parsing, diffing, images	As many threads as CPU cores (at least 2)
`IO`	Blocking I/O: files, legacy blocking SDKs	Up to 64 threads (more on devices with more cores); shares threads with `Default`
`Unconfined`	Almost never in app code	Resumes in whatever thread resumed it

INTERVIEW TRAP

"*`Dispatchers.Default` is a `ForkJoinPool`*." — Not in modern kotlinx.coroutines (last year's version of this challenge said so). `Default` and `IO` share one coroutine scheduler; switching between them often doesn't even change the thread.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Main-safety in practice

```
class ReportRepository(
    private val dao: ReportDao,
    private val cpu: CoroutineDispatcher = Dispatchers.Default,
    private val io: CoroutineDispatcher = Dispatchers.IO,
) {
    // Main-safe: callers never switch threads
    suspend fun export(file: File) {
        val rows = dao.getAll()   // Room suspend: main-safe
        val csv = withContext(cpu) { rows.toCsv() }
        withContext(io) { file.writeText(csv) } // blocking I/O
    }
}
```

- The function that does the heavy work owns the `withContext` — never its caller
- **Inject** dispatchers, so tests can replace them (Q10)

INTERVIEW TRAP

"Wrap every Room or Retrofit call in `withContext(Dispatchers.IO)`." — Their `suspend` functions are already main-safe. The extra switch adds nothing; keep `withContext` for your own blocking or CPU-heavy code.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. launch vs async

	`launch`	`async`
Returns	`Job`	`Deferred<T>` — a `Job` with a result
Result	None — side effects only	`await()` **suspends** until it's ready
Failure	Propagates to the parent	Also propagates to the parent; `await()` rethrows it
Use for	Save, refresh, start collecting	Several independent results in parallel

```
suspend fun loadDashboard(): Dashboard = coroutineScope {
    val user = async { api.user() }    // both start now
    val feed = async { api.feed() }
    Dashboard(user.await(), feed.await())
}   // total time ≈ the slower call, not the sum
```

If either call fails, the other one is cancelled and the exception is rethrown from `loadDashboard()`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. What interviewers check

INTERVIEW TRAP

"An `async` failure stays hidden until someone calls `await()`." — Wrong, and last year's version said it. A failing `async` **child** cancels its parent and siblings immediately. Only a **root** `async` — e.g. `viewModelScope.async { }` — keeps the exception inside the `Deferred`; if nobody awaits it, the error is silently lost.

INTERVIEW TRAP

"`await()` blocks until the result is ready." — It **suspends**. The thread keeps doing other work.

FOLLOW-UP

"What's wrong with `async { repo.load() }.await()`?" — Starting and immediately awaiting is sequential code with extra cost. Call `repo.load()` directly, or use `withContext` if you need another dispatcher.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 4



Mid-Level

Scope & context · Android scopes · Cancellation · Exceptions



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. CoroutineScope and CoroutineContext

```
val scope = CoroutineScope(
    SupervisorJob() + Dispatchers.Default +
        CoroutineName("sync")
)

scope.launch(Dispatchers.IO) {
    // context = scope's context + IO (overrides Default)
    //           + a new Job, child of the scope's Job
}

scope.cancel()  // cancels every coroutine started in it
```

- A **context** is a set of elements, one per key: `Job`, dispatcher, `CoroutineName`, `CoroutineExceptionHandler`
- `+` merges contexts; the right side wins
- A **scope** is just a holder of a context — its `Job` is the parent of everything launched in it

FOLLOW-UP

"*`coroutineScope { }` vs `CoroutineScope(…)`?*" — `coroutineScope` is a suspend function: a child scope that waits for its children and rethrows their failure. `CoroutineScope(…)` creates a new **root** scope that you must cancel yourself.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Breaking the structure by accident

```
// ❌ A new Job becomes the parent: escapes scope.cancel()
scope.launch { launch(Job()) { upload() } }

// ❌ Same problem, and the children are NOT supervised
withContext(SupervisorJob()) { uploadAll() }

// ❌ No owner at all: can't be cancelled or replaced in tests
GlobalScope.launch { upload() }
```

INTERVIEW TRAP

"*`launch(SupervisorJob())` makes this child failure-safe.*" — It doesn't. Passing a `Job` to a builder makes it the **new parent**: the coroutine is detached from its scope, isn't cancelled with it, and its own children still fail together. Use `supervisorScope { }` (Q8).

RULE OF THUMB

Pass dispatchers, names and handlers to `launch` / `withContext` — never a `Job`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Android scopes — who cancels what

Scope	Cancelled when	Use for
`viewModelScope`	The ViewModel is cleared	Screen logic: load, refresh, save
`lifecycleScope`	The lifecycle reaches DESTROYED	One-shot UI work in Activities / Fragments
`repeatOnLifecycle(STARTED)`	Cancelled on STOP, restarted on START	Collecting flows in Views (Day 5)
`rememberCoroutineScope`, `LaunchedEffect`	Leaving the composition	Compose event handlers and effects (Day 8)
Injected application scope	Never — lives with the process	Work that must finish after the user leaves
WorkManager	Survives process death	Guaranteed work (Day 15)

INTERVIEW TRAP

"*`lifecycleScope` is cancelled in `onStop`*." — No, only at DESTROYED (last year's version got this wrong). A collector started with `lifecycleScope.launch` keeps running in the background — that's why `repeatOnLifecycle` exists.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Work that must outlive the screen

```
class DraftRepository(
    private val api: DraftApi,
    private val appScope: CoroutineScope, // injected, app-wide
) {
    suspend fun publish(draft: Draft) {
        // Survives the user leaving the screen
        appScope.launch { api.publish(draft) }.join()
    }
}
```

The caller still waits with `join()`. If the ViewModel is cleared, only the *waiting* is cancelled — the publish continues.

INTERVIEW TRAP

"Use `launchWhenStarted` to pause work in the background." — Deprecated. It only **suspends** the coroutine while upstream producers keep running. `repeatOnLifecycle` cancels and restarts instead.

INTERVIEW TRAP

"Just use `GlobalScope`." — Hard-coded, can't be cancelled as a group and can't be replaced in tests. Inject an application scope instead.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Cancellation is cooperative

`cancel()` doesn't stop a coroutine — it **asks** it to stop. The coroutine notices at its next suspension point or check.

```
suspend fun compress(files: List<File>) = withContext(cpu) {
    for (file in files) {
        ensureActive()        // throws if cancelled
        compressOne(file)     // CPU-heavy, never suspends
    }
}

// A blocking call that should react to cancellation
suspend fun readLegacy(): Data =
    runInterruptible(Dispatchers.IO) {
        legacySdk.readBlocking()
    }
```

- `kotlinx.coroutines` suspend functions (`delay`, `withContext`, `await`, `join`…) are cancellable
- Pure CPU loops and blocking calls are **not** — add `ensureActive()` or `yield()`, or wrap them in `runInterruptible`



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Cleanup, timeouts — and the classic bug

```
scope.launch {
    try {
        upload(file)
    } catch (e: IOException) {
        showError(e)
    } finally {
        withContext(NonCancellable) {
            dao.markUploadIdle()   // suspend call in cleanup
        }
    }
}

val result = withTimeoutOrNull(10.seconds) { api.fetch() }
```

INTERVIEW TRAP

`catch (e: Exception)` also catches `CancellationException` — the coroutine keeps running after it was cancelled. Last year's version said to "catch it and just log it". Always **rethrow** it; the same goes for `runCatching` (Day 3).

FOLLOW-UP

"Why did my `withTimeout` fail silently?" — `TimeoutCancellationException` is a `CancellationException`: inside `launch` it just ends the coroutine — no crash, no handler. Use `withTimeoutOrNull` and handle `null`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Exceptions — where they go

```
// ❌ Never catches: it's thrown inside the coroutine
try {
    scope.launch { error("boom") }
} catch (e: Exception) { }

// ✅ Handle expected failures inside the coroutine
scope.launch {
    try {
        repo.refresh()
    } catch (e: IOException) {
        showError(e)
    }
}

// ✅ Last resort: log on the root coroutine
val handler = CoroutineExceptionHandler { _, e -> log(e) }
viewModelScope.launch(handler) { repo.refresh() }
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. The rules

- **`launch`:** an uncaught exception goes **up** to the parent. At the root it reaches the `CoroutineExceptionHandler` — or the thread's uncaught handler, which **crashes** the app on Android.
- **`async`:** `await()` rethrows the exception — and a child `async` still fails its parent (Q3).
- A `CoroutineExceptionHandler` works only on a **root** coroutine or its scope.
- It runs **after** the coroutine has already failed — it's for logging, not recovery.

INTERVIEW TRAP

"Put a handler on the child that might fail." — `launch(handler)` inside another coroutine is **ignored**: the exception still goes to the parent, and the parent's handler (if any) receives it.

FOLLOW-UP

"Can a `CoroutineExceptionHandler` keep the app alive after a bug?" — Technically it stops the crash; architecturally it's a mistake. See Q11.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 4

→

Senior

coroutineScope vs supervisorScope · Bounded parallel work · Testing coroutines



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. coroutineScope vs supervisorScope

	`coroutineScope` / `Job`	`supervisorScope` / `SupervisorJob`
A child fails	Siblings cancelled, failure rethrown	Only that child fails
Model	All-or-nothing	Independent tasks
Uncaught child exception	Goes to the parent	Goes to the child's handler — or crashes
Example	Profile + its permissions: useless alone	Home screen: news, weather, offers

`viewModelScope` and `lifecycleScope` use a `SupervisorJob`: one failed `launch` doesn't cancel the others.

INTERVIEW TRAP

Supervision applies to **direct children** only. Inside a supervised child, grandchildren started with a normal `launch` still fail together.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. A screen that tolerates partial failure

```
suspend fun loadHome(): Home = supervisorScope {
    val news = async { api.news() }
    val weather = async { api.weather() }
    Home(news.awaitOrNull(), weather.awaitOrNull())
}

suspend fun <T> Deferred<T>.awaitOrNull(): T? = try {
    await()
} catch (e: CancellationException) {
    throw e
} catch (e: IOException) {
    null
}
```

Weather is down? The home screen still shows the news.

INTERVIEW TRAP

"`supervisorScope` *means no crash."* — It only stops a failure from spreading to siblings. An unhandled exception in a supervised `launch` still reaches the uncaught handler and **crashes** the app. Catch inside each child.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Parallel work with limits

"Upload 200 photos — at most 4 at a time, 30 s each, all cancelled if the user leaves."

```
suspend fun uploadAll(photos: List<Photo>) = coroutineScope {
    val permits = Semaphore(4)
    photos.map { photo ->
        async {
            permits.withPermit {
                withTimeout(30.seconds) { api.upload(photo) }
            }
        }
    }.awaitAll()
}
```

- `Semaphore` bounds concurrency; the other coroutines **suspend** waiting for a permit — no thread is blocked
- `coroutineScope` ties all 200 to the caller: cancel the caller and every upload stops
- One failure cancels the rest. Need partial success? `supervisorScope` + a result per item

INTERVIEW TRAP

`photos.forEach { launch { api.upload(it) } }` — 200 requests at once. Coroutines are cheap; the work they start is not. And `withTimeout` **outside** `withPermit` would also count the time spent waiting in the queue.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Testing coroutines — the toolbox

Tool	What it does
`runTest { }`	Runs the test in a `TestScope`; `delay` is skipped using **virtual time**
`StandardTestDispatcher`	Queues new coroutines; run them with `runCurrent()` / `advanceUntilIdle()`
`UnconfinedTestDispatcher`	Starts new coroutines eagerly — on the same virtual clock
`advanceTimeBy()`, `currentTime`	Move and read the virtual clock
`backgroundScope`	For never-ending collectors; cancelled when the test ends
`Dispatchers.setMain()`	Replaces `Main` for `viewModelScope` (Day 3's `MainDispatcherRule`)

INTERVIEW TRAP

"*`UnconfinedTestDispatcher` ignores virtual time.*" — Wrong (last year's version). Both test dispatchers share one `TestCoroutineScheduler`; the only difference is **when** new coroutines start.

INTERVIEW TRAP

`runBlockingTest` and `TestCoroutineDispatcher` are deprecated. Use `runTest` with the test dispatchers above.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Virtual time in action

```
suspend fun <T> retryWithBackoff(block: suspend () -> T): T {
    var wait = 1.seconds
    repeat(2) {
        try { return block() }
        catch (e: IOException) { delay(wait); wait *= 2 }
    }
    return block()
}

@Test fun retriesWithBackoff() = runTest {
    var calls = 0
    val result = retryWithBackoff {
        if (++calls < 3) throw IOException("offline") else "ok"
    }
    assertEquals("ok", result)
    assertEquals(3, calls)
    assertEquals(3_000L, currentTime) // 1 s + 2 s, virtual
}
```

The test finishes in milliseconds: `delay` moves the virtual clock instead of sleeping.

INTERVIEW TRAP

A class that hard-codes `Dispatchers.IO` escapes the test scheduler: its delays are real and its timing is flaky. **Inject** the dispatcher and pass a `StandardTestDispatcher(testScheduler)` in tests.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 4

Architect

Error strategy across layers · Shared mutable state · Under the hood



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. An error strategy for the whole app

```
┌──────────────────────────────────────────────┐
│ Data       catch specific exceptions         │
│            map them to DataError             │
│            rethrow CancellationException     │
├──────────────────────────────────────────────┤
│ ViewModel  Outcome → UiState (message, retry)│
├──────────────────────────────────────────────┤
│ UI         renders state — no try/catch      │
├──────────────────────────────────────────────┤
│ Root       CoroutineExceptionHandler +       │
│            crash reporting: real bugs only   │
└──────────────────────────────────────────────┘
```

- **Expected failures** (offline, 401, validation) are data: model them, show them, offer a retry
- **Bugs** (`NullPointerException`, broken invariants) must fail fast and reach crash reporting

RULE OF THUMB

Catch an exception only in the layer that can **do something** about it. Everything else is a bug — let it propagate.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Expected failures as values

```
sealed interface DataError {
    data object Network : DataError
    data object Unauthorized : DataError
    data class Server(val code: Int) : DataError
}

suspend fun <T> safeCall(block: suspend () -> T): Outcome<T> =
    try {
        Outcome.Ok(block())
    } catch (e: CancellationException) {
        throw e
    } catch (e: IOException) {
        Outcome.Err(DataError.Network)
    } catch (e: HttpException) {
        Outcome.Err(e.toDataError())
    }
```

`Outcome` is a small sealed type (`Ok(value)` / `Err(error)`). Only exceptions you **expect** are caught — anything else is a bug and keeps propagating.

INTERVIEW TRAP

"A global `CoroutineExceptionHandler` that catches everything, so the app never crashes." — Last year's version suggested catching NPEs this way. The handler runs after the coroutine is already dead: the screen is left half-updated and real bugs are hidden. Handle expected errors where they happen; let bugs fail loudly.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Shared mutable state

```
var count = 0
coroutineScope {
    repeat(1_000) {
        launch(Dispatchers.Default) { repeat(100) { count++ } }
    }
}   // count is usually LESS than 100 000: lost updates
```

`count++` is read-modify-write; parallel coroutines overwrite each other. A real case — refresh a token only once:

```
class TokenProvider(private val api: AuthApi) {
    private val mutex = Mutex()
    private var token: Token? = null

    // One refresh at a time; waiting callers reuse the result
    suspend fun validToken(): Token = mutex.withLock {
        token?.takeIf { !it.isExpired() }
            ?: api.refresh().also { token = it }
    }
}
```

INTERVIEW TRAP

"Just use `synchronized`." — A `synchronized` block can't contain a suspension point (the compiler rejects it), and it blocks a thread while waiting. `Mutex` suspends instead.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. The toolbox

Tool	Use when	Watch out
`AtomicInteger` / `AtomicReference` (JVM), atomicfu (KMP)	One value: counters, flags, compare-and-set	Only one variable at a time
`MutableStateFlow.update { }`	Observable state	The lambda may run more than once — keep it pure
`Mutex.withLock { }`	A multi-step critical section that suspends	Not reentrant: nested `withLock` on the same mutex deadlocks
`limitedParallelism(1)`	Confine a non-thread-safe resource to one worker	Atomic only **between** suspension points
One coroutine + a `Channel`	A single owner processes commands in order	The `actor { }` builder (recommended last year) is an obsolete API

INTERVIEW TRAP

"*`limitedParallelism(1)` makes my code atomic.*" — Only between suspension points. `val x = state; delay(…); state = x + 1` still loses updates: another coroutine runs while this one is suspended. A `Mutex` is held across suspension.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Under the hood: CPS and a state machine

```
// You write:
suspend fun loadUser(id: String): User {
    val token = auth.token()        // suspension point 1
    return api.user(id, token)      // suspension point 2
}

// The compiler emits (simplified):
fun loadUser(id: String, cont: Continuation<User>): Any?
// returns a User — or COROUTINE_SUSPENDED
```

- **Continuation-passing style:** every suspend function gets a hidden `Continuation` — "the rest of the work", resumed with `resumeWith(result)`
- The body becomes a **state machine**: a `label` records where to resume; locals needed later are saved in the continuation object, on the heap
- If `auth.token()` returns `COROUTINE_SUSPENDED`, `loadUser` returns too and the thread is free; `resumeWith` later re-enters at the next label

INTERVIEW TRAP

"Coroutines make code faster." — They make **waiting** cheap. CPU-bound work runs no faster; real parallelism is still bounded by the number of cores.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Continuations in your own code

Bridge a one-shot callback API to `suspend` (OkHttp here):

```
suspend fun Call.await(): Response =
    suspendCancellableCoroutine { cont ->
        cont.invokeOnCancellation { cancel() }  // Call.cancel()
        enqueue(object : Callback {
            override fun onResponse(c: Call, r: Response) =
                cont.resume(r)
            override fun onFailure(c: Call, e: IOException) =
                cont.resumeWithException(e)
        })
    }
```

- Streams of callbacks (location, sensors) need `callbackFlow` instead — Day 5

INTERVIEW TRAP

Using `suspendCoroutine` instead: the coroutine **can't be cancelled** while waiting, and the request keeps running. Always prefer `suspendCancellableCoroutine` and cancel the underlying work.

INTERVIEW TRAP

"Thousands of threads cause a `StackOverflowError`." — Last year's version said so. Too many threads exhaust **memory** (thread creation fails with an `OutOfMemoryError`); a `StackOverflowError` comes from recursion that's too deep.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 4 · ARCHITECT

Day 4 — cheat sheet

- `suspend` = may pause without blocking. It does **not** pick a thread.
- Main-safety: the function doing heavy work owns `withContext`. Room and Retrofit are already main-safe.
- `Default` for CPU work, `IO` for blocking I/O — and inject dispatchers.
- `launch` for side effects, `async` for parallel results. A failing child `async` cancels its parent at once.
- Every coroutine has a parent: no `launch(Job())`, no `GlobalScope`. Outliving the screen: app scope or WorkManager.
- `lifecycleScope` dies at DESTROYED — collect with `repeatOnLifecycle`; `launchWhenX` is deprecated.
- Cancellation is cooperative: `ensureActive()`, `runInterruptible`, and always rethrow `CancellationException`.
- `CoroutineExceptionHandler`: root coroutines only, logging only.
- `coroutineScope` = all-or-nothing; `supervisorScope` = independent children — still catch inside them.
- Shared state: atomics → `Mutex` → confinement. `synchronized` can't wrap a suspension. Test with `runTest` and virtual time.

Day 5 is next → Kotlin Flow. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700